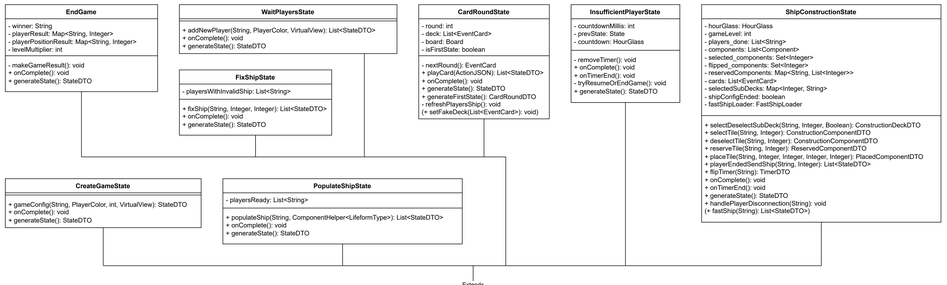
EndGame
- winner: String
- playerResult: Map<String, Integer>
- playerPositionResult: Map<String, Integer>
- levelMultiplier: int
- makeGameResult(): void
+ onComplete(): void
+ generateState(): StateDTO
WaitPlayersState
+ addNewPlayer(String, PlayerColor, VirtualView): List<StateDTO>
+ onComplete(): void
+ generateState(): StateDTO
CardRoundState
- round: int
- deck: List<EventCard>
- board: Board
- isFirstState: boolean
- nextRound(): EventCard
+ playCard(ActionJSON): List<StateDTO>
+ onComplete(): void
+ generateState(): StateDTO
+ generateFirstState(): CardRoundDTO
- refreshPlayersShip(): void
(+ setFakeDeck(List<EventCard>): void)
InsufficientPlayerState
- countdownMillis: int
- prevState: State
- countdown: HourGlass
- removeTimer(): void
+ onComplete(): void
+ onTimerEnd(): void
- tryResumeOrEndGame(): void
+ generateState(): StateDTO
ShipConstructionState
- hourGlass: HourGlass
- gameLevel: int
- players_done: List<String>
- components: List<Component>
- selected_components: Set<Integer>
- flipped_components: Set<Integer>
- reservedComponents: Map<String, List<Integer>>
- cards: List<EventCard>
- selectedSubDecks: Map<Integer, String>
- shipConfigEnded: boolean
- fastShipLoader: FastShipLoader
+ selectDeselectSubDeck(String, Integer, Boolean): ConstructionDeckDTO
+ selectTile(String, Integer): ConstructionComponentDTO
+ deselectTile(String, Integer): ConstructionComponentDTO
+ reserveTile(String, Integer): ReservedComponentDTO
+ placeTile(String, Integer, Integer, Integer, Integer): PlacedComponentDTO
+ playerEndedSendShip(String, Integer): List<StateDTO>
+ flipTimer(String): TimerDTO
+ onComplete(): void
+ onTimerEnd(): void
+ generateState(): StateDTO
+ handlePlayerDisconnection(String): void
(+ fastShip(String): List<StateDTO>)
FixShipState
- playersWithInvalidShip: List<String>
+ fixShip(String, Integer, Integer): List<StateDTO>
+ onComplete(): void
+ generateState(): StateDTO
CreateGameState
+ gameConfig(String, PlayerColor, int, VirtualView): StateDTO
+ onComplete(): void
+ generateState(): StateDTO
PopulateShipState
- playersReady: List<String>
+ populateShip(String, ComponentHelper<LifeformType>): List<StateDTO>
+ onComplete(): void
+ generateState(): StateDTO
Extends
<<Abstract>> State
model: GameModel
+ gameConfig(String, PlayerColor, int, VirtualView): StateDTO
+ addNewPlayer(String, PlayerColor, VirtualView): List<StateDTO>
+ selectDeselectSubDeck(String, Integer, Boolean): ConstructionDeckDTO
+ selectTile(String, Integer): ConstructionComponentDTO
+ deselectTile(String, Integer): ConstructionComponentDTO
+ reserveTile(String, Integer): ReservedComponentDTO
+ placeTile(String, Integer, Integer, Integer, Integer): PlacedComponentDTO
+ playerEndedSendShip(String, Integer): List<StateDTO>
+ flipTimer(String): TimerDTO
+ fixShip(String, Integer, Integer): List<StateDTO>
+ populateShip(String, ComponentHelper<LifeformType>): List<StateDTO>
+ playCard(ActionJSON): List<StateDTO>
+ <<abstract>> onComplete(): void
+ generateState(): StateDTO
+ toString(): String
1
1
<<Abstract>> AbstractShip
+ shipProfiles: Map<Integer, int[][]>
+ shipDimensions: Map<Integer, Pair<Integer, Integer>>
+ shipOffsets: Map<Integer, Pair<Integer, Integer>>
+ grid_rows: int
+ grid_cols: int
# difficultyLevel: int
# areSidesConnected(Connector, Connector): boolean
Extends
Use
Cell
- idx: int
- player: Optional<Player>
- prev: Cell
- next: Cell
+ getIdx(): int
+ getPlayer(): Optional<Player>
+ setPlayer(Player player): void
+ removePlayer(): void
+ getPrevCell(): Cell
+ setPrevCell(Cell): void
+ getNextCell(): Cell
+ setNextCell(Cell): void
+ isEmpty(): boolean
+ toString(): String
<<Abstract>> Board
- boardDimensions: Map<Integer, Pair<Integer, Integer>>
- size: int
- head: Cell
- initialCells: ArrayList<Cell>
- players: List<Player>
- eliminatedPlayer: List<Player>
- level: int
+ getSize(): int
+ setSize(int size): void
+ getHead(): Cell
# getInitialCells(): ArrayList<Cell>
# setInitialCells(ArrayList<Cell> initialCells): void
# addCell(int idx): Cell
+ getPlayers(): List<Player>
+ getEliminatedPlayers(): List<Player>
+ getLevel(): int
# setLevel(int): void
+ newPlayer(Player player): Board
+ eliminatePlayer(Player player): Board
# addCell(int): Cell
+ <<abstract>> buildBoard(): void
+ addPlayerToBoard(Player): void
+ movePlayerForward(Player, int): void
+ movePlayerBackwards(Player, int): void
+ validatePlayersPositions(): void
+ generateState(): BoardJSON
- getBoardTitleWidget(): WidgetTUI
- getBoardInfoWidget(): WidgetTUI
+ generateWidget(): WidgetTUI
GameModel
- level: int;
- board: Board;
- deck: List<EventCard>;
- resourceBank: ResourceBank;
- numPlayers: int;
- players: Map<String, Player>;
- playerVirtualViews: Map<String, VirtualView>;
- currentState: State;
- queueHandler: Queue;
- random: Random;
+ getCurrentState(): State
+ setCurrentState(State): void
+ generateState(): StateDTO
+ disconnectClient(String): List<StateDTO>
+ getDisconnectedPlayers(): List<String>
+ reconnectClient(String, VirtualView): List<StateDTO>
- generateDeck(): void
+ gameConfig(String, PlayerColor, int, VirtualView): StateDTO
- createBoard(): void
+ addNewPlayer(String, PlayerColor, VirtualView): List<StateDTO>
+ selectDeselectSubDeck(String, Integer, Boolean): ConstructionDeckDTO
+ selectTile(String, Integer): ConstructionComponentDTO
+ deselectTile(String, Integer): ConstructionComponentDTO
+ reserveTile(String, Integer): ReservedComponentDTO
+ placeTile(String, Integer, Integer, Integer, Integer): PlacedComponentDTO
+ playerEndedSendShip(String, Integer): List<StateDTO>
+ flipTimer(String): TimerDTO
+ fixShip(String, Integer, Integer): List<StateDTO>
+ populateShip(String, ComponentHelper<LifeformType>): List<StateDTO>
+ playCard(ActionJSON): List<StateDTO>
- getGameLevel(): int
~ setGameLevel(int): void
- getGameDeck(): List<EventCard>
~ getNumPlayers(): int
~ setGamePlayersNumber(int): void
- getPlayers(): Map<String, Player>
+ getAvailableColors(): List<String>
~ addPlayer(String, PlayerColor): void
~ addPlayerToBoard(String): void
~ getBoard(): Board
~ broadCastUpdate(Answer): void
- sendUpdateWithRetries(VirtualView, Answer): void
(+ fastShip(String): List<StateDTO>)
1
1
1..4
Player
- color: PlayerColor;
- nickname: String;
- ship: Ship;
- cursor: int;
- credits: int;
- lostPieces: int;
- cell: Cell;
- connected: boolean;
- lost: boolean;
+ getNickname(): String
+ getColorToString(): String
+ getColor(): PlayerColor
+ getCursor(): int
+ setCursor(int): Player
+ getCredits(): int
+ setCredits(int): Player
+ addCredits(int): Player
+ getShip(): Ship
+ getCurrentCell(): Cell
+ setCurrentCell(Cell): Player
+ getLostPieces(): int
+ setLostPieces(int): Player
+ addLostPieces(int): Player
+ eliminate(): Player
+ isEliminated(): boolean
+ isConnected(): boolean
+ setConnected(boolean): Player
1
1
Ship
- components: Component[][]
- core: Cabin
- purpleAlienPosition: Cabin
- brownAlienPosition: Cabin
- batteryList: List<Battery>
- cabinList: List<Cabin>
- cannonList: List<Cannon>
- engineList: List<Engine>
- shieldList: List<Shield>
- storageList: List<Storage>
- vitalList: List<Vital>
+ getCore(): Cabin
+ getPurpleAlienPosition(): Cabin
+ getBrownAlienPosition(): Cabin
+ generateComponentSublist(): void
+ getDifficultyLevel(): int
+ getBatteryList(): List<Battery>
+ getCabinList(): List<Cannon>
+ getCannonList(): List<Cannon>
+ getEngineList(): List<Engine>
+ getShieldList(): List<Shield>
+ getStorageList(): List<Storage>
+ getVitalList(): List<Vital>
+ getDoubleEngines(): List<Engine>
+ getDoubleCannons(): List<Cannon>
+ getAvailableEnergy(): int
+ consumeEnergy(List<CoordinatePair>): void
+ getAllLifeforms(): List<Lifeform>
+ isShipPopulated(): boolean
+ addLifeformToCabin(int, int, LifeformType): void
+ removeLifeformFromCabin(int, int, LifeformType): void
+ getFirePower(List<CoordinatePair>): float
+ getEnginePower(List<CoordinatePair>): int
+ activateComponents(List<Pair<CoordinatePair, CoordinatePair>>): List<Pair<CoordinatePair, CoordinatePair>
+ getExposedConnectorAmount(): int
+ getAvailableStorageSpace(): int
+ getAllItems(): List<Item>
+ getAllItemValue(): int
+ getGridRow(int): Component[]
+ getGridColumn(int): Component[]
+ validateShip(): boolean
+ recreateShipGrid(): void
- initGrid(): Component[][]
+ traverse(Consumer<Component>): void
+ getNearestComponents(Component): Component[]
getNearestReachableComponents(Component): Component[]
+ addComponent(Component, int, int): void
+ removeSingleComponent(int, int): void
+ removeComponent(int, int): List<Component>
+ getComponent(int, int): Component
+ generateState(): List<Map<String, Object>>
<<Enum>> LifeformType
- requiredSpace: int
- attackBoost: int
- powerBoost: int
+ getRequiredSpace(): int
+ getAttackBoost(): int
+ getPowerBoost(): int
+ toString(): String
+ getImagePath(): String
+ fromString(String): LifeformType
Use
Lifeform
- lifeformType: LifeformType
+ getLifeformType: LifeformType
+ getRequiredSpace(): int
+ getAttackBoost(): int
+ getPowerBoost(): int
<<Enum>> Connector
+ fromOrdinal(int): Connector
Use
<<Abstract>> Component
- row: int
- col: int
- sides: Connector[]
- direction: int
- path: String
- id: int
+ getTypeId: int
+ requiresEnergy: boolean
+ check(Component[]): boolean
+ getDirection(): int
+ getPosition(): int[]
+ setPosition(int, int): void
+ setRotation(int): Component
+ rotateLeft(): Component
+ rotateRight(): Component
+ getLeftSide(): Connector
+ getRightSide(): Connector
+ getTopSide(): Connector
+ getBottomSide(): Connector
+ getId(): int
+ setId(): int
+ toMap(): Map<String, Object>
+ getComponentScreen(): List<String>
Extends
ItemColor
- value: int
+ getValue(): int
+ toString(): String
+ getANSIColor(): String
+ getImagePath(): String
Use
Item
- color: ItemColor
+ getColor(): ItemColor
+ getValue(): int
+ toString(): String
<<Enum>> VitalType
Use
Battery
+ alias: String
- maxAvailability: int
- available: int
+ getAvailability(): int
+ getMaxAvailability(): int
+ setAvailability(int): void
+ useBattery(int): void
Cabin
+ alias: String
- isCore: boolean
- inhabitants: List<Lifeform>
+ isCore(): boolean
+ getInhabitants(): List<Lifeform>
+ getAvailableSpace(): int
+ addInhabitant(Lifeform): void
+ removeInhabitant(Lifeform): void
1..n
Cannon
+ alias: String
- force: int
+ getFirePower(): float
Engine
+ alias: String
- speed: int
+ getSpeed(): int
Shield
+ alias: String
+ getCoveredSide(): int[]
Storage
+ alias: String
- capacity: int
- isSpecialStorage: boolean
- storedItems: List<Item>
+ getCapacity(): int
+ isSpecialStorage(): boolean
+ getStoredItems(): List<Item>
+ storeItem(Item): void
+ removeItem(Item): void
+ availableSpace(): int
1..n
Structural
+ alias: String
Vital
+ alias: String
- vitalType: VitalType
+ getVitalType(): VitalType
Use
<<Enum>> CardType
Use
<<Abstract>> EventCard
cardTypeId: int
cardName: String
cardLevel: int
board: Board
uniqueCardId: int
path: String
players: List<Player>
currentPlayer: Optional<Player>
hasBeenUsed: boolean
hasBeenActivated: boolean
+ getCardTypeId(): int
+ getCardName(): String
+ getCardLevel(): int
# getBoard(): Board
+ getUniqueCardId(): int
# cardUsed(): void
+ initCardPlayers(): void
# bonusEffect(): void
# bonusEffect(ActionJSON): void
# malusEffect(): void
# malusEffect(ActionJSON): void
# getNextPlayer(): Optional<Player>
# getCurrentPlayer(): Optional<Player>
# activateCard(): void
# hasBeenActivated(): boolean
+ hasFinished(): boolean
+ useCard(ActionJSON): EventCard
+ generateState(): CardStateJSON
+ generateStaticState(): CardStateJSON
# initStateFlags(): void
# setUpdatedPositionsIfNecessary(): void
# setUpdatedEliminatedPlayersIfNecessary(): void
# setUpdatedCreditsIfNecessary(): void
# setUpdatedResourcesIfNecessary(): void
# setUpdatedTakenResourcesIfNecessary(): void
# setUpdatedRemovedComponentsIfNecessary(): void
# setUpdatedRemovedLifeformsIfNecessary(): void
# setUpdatedRemovedBatteriesIfNecessary(): void
# setUpdatedLostPiecesIfNecessary(): void
Extends
AbandonedShip
- requiredCrew: int
- movementStep: int
- givenCredits: int
- lifeformsToBeRemoved: List<ComponentHelper<Lifeform>>
- removedLifeforms: Map<String, List<ComponentHelper<Lifeform>>>
- eliminatedPlayer: List<String>
- updatedPositions: Map<String, Integer>
- updatedCredits: Map<String, Integer>
- prevPlayerNickname: String
- hasBeenUsedByPlayer: boolean
- playersThatCanUseTheCard: List<String>
AbandonedStation
- requiredCrew: int
- movementSteps: int
- givenItems: List<Item>
- resourceBank: ResourceBank
- resourcesToDropOff: List<ComponentHelper<ItemColor>>
- resourcesToTake: List<ComponentHelper<ItemColor>>
- playersThatCanUseTheCard: List<String>
- droppedResources: Map<String, List<ComponentHelper<ItemColor>>>
- takenResources: Map<String, List<ComponentHelper<ItemColor>>>
- updatedPositions: Map<String, Integer>
- eliminatedPlayers: List<String>
- prevPlayerNickname: String
Epidemy
- removedLifeforms: Map<String, List<ComponentHelper<LifeformType>>>
+ useCard(): void
Stardust
- updatedPositions: Map<String, Integer>
- eliminatedPlayers: List<String>
Smugglers
- requiredFirepower: int
- movementSteps: int
- redItems: int
- yellowItems: int
- greenItems: int
- blueItems: int
- resourceBank: ResourceBank
- takenItems: int
- defeatedPlayers: List<String>
- isPlayerDefeated: boolean
- updatedPositions: Map<String, Integer>
- droppedResources: Map<String, List<ComponentHelper<ItemColor>>>
- takenResources: Map<String, List<ComponentHelper<ItemColor>>>
- removedBatteries: Map<String, List<CoordinatePair>>
- eliminatedPlayers: List<String>
- prevPlayerNickname: String
- countOccurrences(List<ItemColor>): Map<ItemColor, Integer>
ResourceBank
- resources: Map<ItemColor, Integer>
+ resetResourcesQuantity(Map<ItemColor, Integer>): void
+ getResources(): Map<ItemColor, Integer>
+ getResourceAvailabilityFromColor(ItemColor): int
+ addResourceToBank(ItemColor): void
+ removeResourceFromBank(ItemColor): void
+ addResourceToPlayerFromBank(Player, ItemColor, int, int): ResourceBank
+ addResourceToBankFromPlayer(Player, ItemColor, int, int): ResourceBank
Use
WarZone
- resourceBank: ResourceBank
- movementSteps: int
- requiredCrew: int
- requiredItems: int
- shootingSequence: List<PlasmaShot>
- cardActions: List<WarZoneActionConsequencePair>
- current_action: int
- affectedPlayer: Optional<Player>
- playersEnginePower: Map<Player, Integer>
- playersFirePower: Map<Player, Float>
- current_plasmaShot: int
- random: Random
- diceResult: int
- previousPlayerRemovedComponents: List<Component>
- prevPlayer: String
- removedComponents: Map<String, List<Map<String, Object>>>
- droppedResources: Map<String, List<ComponentHelper<ItemColor>>>
- removedBatteries: Map<String, List<CoordinatePair>>
- updatedPositions: Map<String, Integer>
- removedLifeforms: Map<String, List<ComponentHelper<LifeformType>>>
- eliminatedPlayer: List<String>
- lostPieces: Map<String, Integer>
- targetPlayer: String
- prevPlayerNickname: String
- handleFirePower(WarZoneActionConsequencePair, WarZoneJSON): WarZone
- handleEnginePower(WarZoneActionConsequencePair, WarZoneJSON): WarZone
- handleHumansWarZoneActionConsequencePair, WarZoneJSON): WarZone
- applyConsequences(Player, WarZoneJSON): WarZone
- handleLoselItems(Player, WarZoneJSON): WarZone
- countOccurrences(List<ItemColor>): Map<ItemColor, Integer>
- handleRequiredCrew(Player, WarZoneJSON): WarZone
- handlePlasmaShot(Player, WarZoneJSON): WarZone
- generateDiceResult(): int
+ forceDiceThrow(int): void
<<Enum>> WarZoneConsequence
+ fromInteger(int): WarZoneConsequence
+ toString(): String
Use
WarZoneActionConsequencePair
- action: WarZoneAction
- consequence: WarZoneConsequence
+ getAction(): WarZoneAction
+ getConsequence(): WarZoneConsequence
+ toString(): String
Use
<<Enum>> WarZoneAction
+ fromInteger(int): WarZoneAction
+ toString(): String
Pirates
- requiredFirepower: int
- givenCredits: int
- movementSteps: int
- shootingSequence: List<List<Integer>>
- random: Random
- diceThrowResult: int
- plasmaShotIndex: int
- currentPlasmaShot: Map<String, Integer>
- shotSize: int
- shotDirection: int
- firstRound: boolean
- playersToHit: List<Player>
- previousPlayerRemovedComponents: List<Component>
- eliminatedPlayers: List<String>
- updatedPositions: Map<String, Integer>
- updatedCredits: Map<String, Integer>
- removedComponents: Map<String, List<Map<String, Object>>>
- removedBatteries: Map<String, List<>>
- lostPieces: Map<String, Integer>
- removedLifeforms: Map<String, List<ComponentHelper<LifeformType>>>
- prevPlayerNickname: String
+ setDiceThrowResult(int): void
Slavers
- requiredFirepower: int
- movementSteps: int
- givenCredits: int
- takenCrew: int
- eliminatedPlayers: List<String>
- updatedPositions: Map<String, Integer>
- updatedCredits: Map<String, Integer>
- removedBatteries: Map<String, List<CoordinatePair>>
- removedLifeforms: Map<String, List<ComponentHelper<LifeformType>>>
- isPlayerDefeated: boolean
- prevPlayerNickname: String
MeteorShower
- meteorSequence: List<Meteor>
- random: Random
- currMeteorIndex: int
- diceThrowResult: int
- prevPlayerRemovedComponents: List<Component>
- prevPlayer: String
- removedComponents: Map<String, List<Map<String, Object>>>
- removedBatteries: Map<String, List<CoordinatePair>>
- eliminatedPlayers: List<String>
- lostPieces: Map<String, Integer>
- removedLifeforms: Map<String, List<ComponentHelper<LifeformType>>>
+ setDiceThrowResult(int): void
VisitPlanets
- resourceBank: ResourceBank
- updatedPositions: Map<String, Integer>
- droppedResources: Map<String, List<ComponentHelper<ItemColor>>>
- takenResources: Map<String, List<ComponentHelper<ItemColor>>>
- eliminatedPlayers: List<String>
- itemsPerPlanet: Map<Integer, Map<ItemColor, Integer>>
- playersChosenPlanets: Map<Integer, Player>
- itemsToDrop: List<ComponentHelper<ItemColor>>
- itemsToTake: List<ComponentHelper<ItemColor>>
- movementSteps: int
- chosenPlanetIntex: int
- prevPlayerNickname: String
OpenSpace
- playerPowerResult: Map<String, Player>
- updatedPositions: Map<String, Integer>
- eliminatedPlayers: List<String>
- removedBatteries: Map<String, List<CoordinatePair>>
- prevPlayerNickname: String
+ method(type): type
PlasmaShot
- size: int
- orientation: int
+ getSize(): int
+ getOrientation(): int
Meteor
- size: int
- orientation: int
+ getSize(): int
+ getOrientation(): int
Use
Use
Use
Use